

Tier 7: Causal Confounder Adjustment

Contents

Overview	1
Load Libraries	1
Locate and Read Data	1
Build the Sepsis-3 Cohort with Antibiotic Timing	2
Collapse High-Cardinality Covariates	2
LASSO Covariate Selection	2
Crude (Unadjusted) Estimate	3
Propensity-Score Matching	3
Covariate Balance Diagnostics	4
Inverse-Probability Weighting	4
Doubly Robust / Augmented-IPW Estimator	4
Summary of Effect Estimates	5

Overview

This report performs a confounder-adjusted causal analysis of whether early antibiotic administration (within 3 hours of ICU admission, following the Surviving Sepsis Campaign hour-3 bundle threshold) lowers in-hospital mortality among Sepsis-3 patients. Confounding by indication is the core concern: sicker patients may be treated faster (or slower, if sepsis is not yet recognized), so a naive comparison of early- versus delayed-antibiotic mortality is biased. Four estimators are compared, from least to most robust to model misspecification: (1) crude unadjusted, (2) propensity-score matching, (3) inverse-probability weighting, and (4) augmented-IPW (doubly robust). The adjustment set is chosen by LASSO rather than by hand.

Load Libraries

We load `glmnet` for LASSO covariate selection, `MatchIt` for propensity-score matching, `WeightIt` for inverse-probability weighting, `cobalt` for balance diagnostics, and `sandwich` for robust standard errors.

```
suppressPackageStartupMessages({  
  library(glmnet); library(MatchIt); library(WeightIt); library(cobalt); library(sandwich)  
})  
set.seed(486649)
```

Locate and Read Data

We read the analysis matrix (containing Sepsis-3 labels, SOFA components, demographics, and mortality) and the suspected-infection file (containing antibiotic administration times). Timestamps are parsed with an explicit format to avoid silent truncation when even one element is malformed.

```
find_data <- function(f) {  
  cands <- c(file.path("../..//processed_data", f),  
             file.path("processed_data", f), f)  
  hit <- cands[file.exists(cands)][1]  
  if (is.na(hit)) stop("cannot find ", f); hit  
}
```

```

OUT <- dirname(find_data("analysis_matrix.csv"))
FIG <- if (dir.exists("figure")) "figure" else { dir.create("figure"); "figure" }

am <- read.csv(find_data("analysis_matrix.csv"))
si <- read.csv(find_data("suspected_infection.csv"))
# NB: as.POSIXct() on a full character VECTOR silently falls back to a
# date-only format (dropping time-of-day for every row) if even one element
# fails to match "%Y-%m-%d %H:%M:%S" -- an R analogue of the pandas Arrow
# CSV round-trip gotcha documented in GENAI/HANDOFF.md. Force the format
# explicitly so a single malformed timestamp can't truncate the whole column.
parse_dt <- function(x) as.POSIXct(x, format = "%Y-%m-%d %H:%M:%S", tz = "UTC")
am$intime <- parse_dt(am$intime)
si$t_abx <- parse_dt(si$t_abx)

```

Build the Sepsis-3 Cohort with Antibiotic Timing

We filter to Sepsis-3 patients, merge in the antibiotic time, and create the binary exposure variable indicating whether antibiotics were given within 3 hours of ICU admission.

```

# ---- Cohort: Sepsis-3 patients with a recorded antibiotic time ----
d <- am[am$sepsis3 == "True" | am$sepsis3 == TRUE, ]
d <- merge(d, si[, c("stay_id", "t_abx")], by = "stay_id")
d$hours_to_abx <- as.numeric(difftime(d$t_abx, d$intime, units = "hours"))
d$early_abx <- as.integer(d$hours_to_abx <= 3)
d$mortality <- d$hospital_expire_flag
cat(sprintf("Sepsis-3 cohort with antibiotic timing: %d stays (%.0f%% early <=3h), mortality %.1f%%\n",
           nrow(d), 100 * mean(d$early_abx), 100 * mean(d$mortality)))

```

Sepsis-3 cohort with antibiotic timing: 24203 stays (48% early <=3h), mortality 18.7%

Collapse High-Cardinality Covariates

Admission type, care unit, and race are collapsed into a small number of clinically sensible groups so that propensity-score matching and weighting do not struggle with hundreds of near-empty dummy categories.

```

# ---- Collapse high-cardinality covariates into a few clinically sensible ----
# groups, so PSM/IPW aren't fighting hundreds of near-empty dummy categories.
d$admission_grp <- with(d, ifelse(grepl("EMER|URGENT", admission_type), "EMERGENT",
                                   ifelse(grepl("ELECTIVE|SURGICAL SAME DAY", admission_type), "ELECTIVE",
                                           "OBSERVATION"))))
d$careunit_grp <- with(d, ifelse(grepl("Medical", first_careunit), "MEDICAL",
                                   ifelse(grepl("Cardiac|Coronary", first_careunit), "CARDIAC",
                                           ifelse(grepl("Neuro", first_careunit), "NEURO",
                                                  ifelse(grepl("Surg|Trauma", first_careunit), "SURGICAL", "OTHER"))))))
top_race <- names(sort(table(d$race), decreasing = TRUE))[1:4]
d$race_grp <- ifelse(d$race %in% top_race, d$race, "OTHER/UNKNOWN")
for (v in c("admission_grp", "careunit_grp", "race_grp"))
  d[[v]] <- factor(d[[v]])
d$female <- as.integer(d$gender == "F")

```

LASSO Covariate Selection

Following Guo et al.'s design, we use LASSO penalized logistic regression on the outcome model (mortality) to select confounders from a set of candidate covariates. The exposure variable `early_abx` is forced in unpenalized so it cannot be removed by the penalty.

```

# ---- LASSO covariate selection (Guo et al.'s design) -----
# Candidate confounders: demographics + case-mix + baseline organ dysfunction
# (SOFA components). Select via LASSO on the OUTCOME model (mortality), not
# by hand, keeping only covariates that survive penalization; `early_abx`
# itself is forced in unpenalized (penalty.factor=0) since it's the exposure
# of interest, not a candidate for removal.
cand_form <- ~ early_abx + age + female + admission_grp + careunit_grp + race_grp +
  sofa_resp + sofa_coag + sofa_liver + sofa_cardio + sofa_cns + sofa_renal
Xcand <- model.matrix(cand_form, data = d)[, -1] # drop intercept
pen <- ifelse(colnames(Xcand) == "early_abx", 0, 1)
cv <- cv.glmnet(Xcand, d$mortality, family = "binomial", alpha = 1,
  penalty.factor = pen, nfolds = 10)
sel <- rownames(coef(cv, s = "lambda.min"))[which(coef(cv, s = "lambda.min")[, 1] != 0)]
sel <- setdiff(sel, c("(Intercept)", "early_abx"))
# Map selected DUMMY columns back to their parent covariate (need whole
# factors, not individual levels, for matching/weighting formulas).
parents <- c("age", "female", "admission_grp", "careunit_grp", "race_grp",
  "sofa_resp", "sofa_coag", "sofa_liver", "sofa_cardio", "sofa_cns", "sofa_renal")
kept <- parents[sapply(parents, function(p) any(grepl(paste0("^", p), sel)) || p %in% sel)]
if (length(kept) < 3) kept <- parents # fallback: LASSO pruned too aggressively
cat(sprintf("\nLASSO-selected confounder set (%d/%d candidates): %s\n",
  length(kept), length(parents), paste(kept, collapse = " ")))

##
## LASSO-selected confounder set (11/11 candidates): age, female, admission_grp, careunit_grp, race_grp
adj_form <- as.formula(paste("early_abx ~", paste(kept, collapse = " + ")))

```

Crude (Unadjusted) Estimate

We compute the unadjusted odds ratio and risk difference as a baseline before applying any confounder adjustment.

```

# ---- (0) Crude (unadjusted) estimate -----
crude_glm <- glm(mortality ~ early_abx, data = d, family = binomial)
crude_or <- exp(coef(crude_glm)["early_abx"])
crude_rd <- mean(d$mortality[d$early_abx == 1]) - mean(d$mortality[d$early_abx == 0])

```

Propensity-Score Matching

We perform 1:1 nearest-neighbour propensity-score matching with a caliper of 0.2 standard deviations. The treatment effect on the matched sample (ATT) is estimated with cluster-robust standard errors grouped by matched pair.

```

# ---- (1) Propensity-score matching (1:1 nearest-neighbour, caliper 0.2 SD) --
m.out <- matchit(adj_form, data = d, method = "nearest", distance = "glm", caliper = 0.2)
md <- match.data(m.out)
# ATT on the matched sample: cluster-robust (by matched pair/subclass) SE.
psm_glm <- glm(mortality ~ early_abx, data = md, weights = weights, family = binomial)
psm_se <- sqrt(diag(vcovCL(psm_glm, cluster = md$subclass)))
psm_or <- exp(coef(psm_glm)["early_abx"])
psm_ci <- exp(coef(psm_glm)["early_abx"] + c(-1.96, 1.96) * psm_se["early_abx"])

```

Covariate Balance Diagnostics

We assess how well matching balanced the covariates by computing standardized mean differences before and after matching. Results are saved as a CSV and visualized in a Love plot.

```
# ---- Covariate balance: unmatched vs matched (love plot + SMD table) ----
bal <- bal.tab(m.out, un = TRUE, m.threshold = 0.1)
bal_df <- as.data.frame(bal$Balance)
bal_df$covariate <- rownames(bal_df)
write.csv(bal_df[, c("covariate", grep("Diff", names(bal_df), value = TRUE))],
          file.path(OUT, "tier7_covariate_balance.csv"), row.names = FALSE)
png(file.path(FIG, "tier7_balance_love_plot.png"), width = 1200, height = 900, res = 150)
print(love.plot(m.out, stats = "mean.diffs", threshold = 0.1, abs = TRUE,
               var.order = "unadjusted",
               title = "Covariate balance: before vs after PS matching"))
dev.off()

## pdf
## 2
```

Inverse-Probability Weighting

We estimate ATE weights from the propensity score and fit a weighted outcome model with heteroscedasticity-consistent standard errors.

```
# ---- (2) Inverse-probability weighting (ATE weights) ----
w.out <- weightit(adj_form, data = d, method = "ps", estimand = "ATE")
ipw_glm <- glm(mortality ~ early_abx, data = d, weights = w.out$weights, family = binomial)
ipw_se <- sqrt(diag(vcovHC(ipw_glm, type = "HC0")))
ipw_or <- exp(coef(ipw_glm)["early_abx"])
ipw_ci <- exp(coef(ipw_glm)["early_abx"] + c(-1.96, 1.96) * ipw_se["early_abx"])
```

Doubly Robust / Augmented-IPW Estimator

The AIPW estimator is consistent for the ATE if either the propensity model or the outcome model is correctly specified (van der Laan and Robins, 2003). We fit separate outcome regressions under treatment and control, then combine them with the propensity weights. A nonparametric bootstrap (200 replicates) provides the confidence interval, refitting all three models in each replicate.

```
# ---- (3) Doubly robust / augmented-IPW estimator ----
# Consistent for the ATE if EITHER the propensity model OR the two outcome
# regressions (mu1, mu0) are correctly specified (van der Laan & Robins 2003).
ps <- w.out$ps
outcome_form <- as.formula(paste("mortality ~", paste(kept, collapse = " + ")))
mu1_fit <- glm(outcome_form, data = d[d$early_abx == 1, ], family = binomial)
mu0_fit <- glm(outcome_form, data = d[d$early_abx == 0, ], family = binomial)
mu1 <- predict(mu1_fit, newdata = d, type = "response")
mu0 <- predict(mu0_fit, newdata = d, type = "response")
aipw_est <- function(dd, ps_, mu1_, mu0_) {
  psi1 <- mean(mu1_ + dd$early_abx * (dd$mortality - mu1_) / ps_)
  psi0 <- mean(mu0_ + (1 - dd$early_abx) * (dd$mortality - mu0_) / (1 - ps_))
  psi1 - psi0
}
aipw_rd <- aipw_est(d, ps, mu1, mu0)

# Nonparametric bootstrap CI for the AIPW risk difference (refits propensity +
```

```

# both outcome models each replicate, so the CI reflects all three model fits).
cat("bootstrapping AIPW CI (200 reps) ...\n")

## bootstrapping AIPW CI (200 reps) ...
B <- 200
boot_rd <- numeric(B)
n <- nrow(d)
for (b in 1:B) {
  idx <- sample(n, n, replace = TRUE)
  db <- d[idx, ]
  wb <- tryCatch(weightit(adj_form, data = db, method = "ps", estimand = "ATE"),
    error = function(e) NULL)
  if (is.null(wb)) { boot_rd[b] <- NA; next }
  m1b <- tryCatch(glm(outcome_form, data = db[db$early_abx == 1, ], family = binomial),
    error = function(e) NULL)
  m0b <- tryCatch(glm(outcome_form, data = db[db$early_abx == 0, ], family = binomial),
    error = function(e) NULL)
  if (is.null(m1b) || is.null(m0b)) { boot_rd[b] <- NA; next }
  boot_rd[b] <- aipw_est(db, wb$ps, predict(m1b, newdata = db, type = "response"),
    predict(m0b, newdata = db, type = "response"))
}
aipw_ci <- quantile(boot_rd, c(0.025, 0.975), na.rm = TRUE)

```

Summary of Effect Estimates

We compile all four estimators (crude, PSM, IPW, and AIPW) into a single summary table with point estimates and confidence intervals where available, then save the results.

```

# ---- Summary table -----
effects <- data.frame(
  method = c("Crude (unadjusted)", "PSM (1:1 nearest, ATT)",
    "IPW (ATE weights)", "AIPW / doubly robust (ATE)"),
  estimand = c("RD & OR", "OR (matched)", "OR (weighted)", "RD"),
  risk_difference = c(crude_rd, NA, NA, aipw_rd),
  rd_ci_low = c(NA, NA, NA, aipw_ci[1]), rd_ci_high = c(NA, NA, NA, aipw_ci[2]),
  odds_ratio = c(crude_or, psm_or, ipw_or, NA),
  or_ci_low = c(NA, psm_ci[1], ipw_ci[1], NA),
  or_ci_high = c(NA, psm_ci[2], ipw_ci[2], NA))
effects[, -(1:2)] <- round(effects[, -(1:2)], 4)
cat("\nAntibiotic-timing effect on in-hospital mortality, by method:\n")

```

```

##
## Antibiotic-timing effect on in-hospital mortality, by method:
print(effects, row.names = FALSE)

```

##	method	estimand	risk_difference	rd_ci_low	rd_ci_high
##	Crude (unadjusted)	RD & OR	-0.0187	NA	NA
##	PSM (1:1 nearest, ATT)	OR (matched)	NA	NA	NA
##	IPW (ATE weights)	OR (weighted)	NA	NA	NA
##	AIPW / doubly robust (ATE)	RD	-0.0191	-0.0284	-0.0107
##	odds_ratio	or_ci_low	or_ci_high		
##	0.8840	NA	NA		
##	0.8994	0.8410	0.9617		
##	0.8894	0.8325	0.9501		

```
##          NA          NA          NA
write.csv(effects, file.path(OUT, "tier7_effect_estimates.csv"), row.names = FALSE)
cat("\nTier 7 (confounder-adjusted causal analysis) complete.\n")

##
## Tier 7 (confounder-adjusted causal analysis) complete.
```